REPORTE DE AUDITORIA DE SEGURIDAD

API MVC - Analisis de Codigo Estatico y Dinamico

Materia: Ciberseguridad - Tecmilenio

Agente Auditor: Joaquin - Rakesh Cibersecurity

Fecha: 12 de noviembre de 2026, 18:35 hrs

Clasificacion: USO ACADEMICO

1. RESUMEN EJECUTIVO

Se realizo una auditoria de seguridad sobre la API MVC ubicada en /home/rodrigo/Escritorio/CIBERSEGURIDAD/API/api_mvc/. La API esta construida con el modulo estandar http.server de Python 3.14, implementa el patron MVC (Modelo-Vista-Controlador) y expone tres endpoints REST: /api/salud, /api/productos y /api/privado (este ultimo protegido por Bearer token).

La auditoria combino analisis de codigo estatico (revision manual de cada archivo fuente) con pruebas dinamicas contra la API en ejecucion (http://127.0.0.1:5000). Se identificaron 7 hallazgos clasificados por severidad.

Resumen de hallazgos

Severidad	Cantidad	Estado
MEDIA	3	Confirmados
BAJA	2	Confirmados
INFO	2	Observaciones

No se encontraron vulnerabilidades de severidad CRITICA ni ALTA. La API, al ser un ejercicio academico con datos ficticios y token fijo, presenta un nivel de riesgo aceptable para su proposito educativo.

2. ALCANCE DE LA AUDITORIA

2.1 Objetivo auditado

API REST MVC construida con http.server de Python (sin framework externo).

> Ruta: /home/rodrigo/Escritorio/CIBERSEGURIDAD/API/api_mvc/
> Servidor: ThreadingHTTPServer en 127.0.0.1:5000
> Lenguaje: Python 3.14.10
> Entorno virtual: .venv/ (activado para ejecucion)

2.2 Archivos analizados

> app.py - Punto de entrada, servidor HTTP
> controllers/api_controller.py - Enrutamiento y logica de dispatch
> models/producto.py - Modelo de datos (productos ficticios)
> views/json_view.py - Serializacion JSON
> security/token_auth.py - Autenticacion por Bearer token

2.3 Endpoints probados

> GET /api/salud - Health check (publico)
> GET /api/productos - Lista de productos (publico)
> GET /api/privado - Recurso protegido (requiere Bearer token)

2.4 Metodologia

> Analisis estatico: revision linea por linea del codigo fuente
> Analisis dinamico: pruebas con curl contra la API en ejecucion
> Pruebas de autenticacion: token valido, invalido, ausente, malformado
> Pruebas de rutas: path traversal, rutas inexistentes, metodos no permitidos
> Pruebas de inyeccion: SQL injection via query string, header injection

> Pruebas de headers: verificacion de cabeceras de seguridad
> Pruebas de CORS: verificacion de politica de origen cruzado

3. HALLAZGOS DETALLADOS

MEDIA **VUL-001 - Token de autenticacion hardcodeado en codigo fuente**

Descripcion:

El archivo security/token_auth.py contiene el token de autenticacion como constante en texto plano: TOKEN_DEMO = 'clase13-token-demo'. Cualquier persona con acceso al codigo fuente puede autenticarse. En un entorno real, esto constituiria una exposicion de credenciales (CWE-798).

Evidencia:

```
TOKEN_DEMO = "clase13-token-demo"  # Linea 3 de token_auth.py
Archivo: security/token_auth.py
```

Recomendacion:

Usar variables de entorno (os.environ['API_TOKEN']) o un gestor de secretos. Nunca incluir credenciales en el codigo fuente.

MEDIA **VUL-002 - Ausencia de headers de seguridad HTTP**

Descripcion:

Las respuestas de la API no incluyen headers de seguridad estandar: X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, Content-Security-Policy, ni X-XSS-Protection. Esto podria facilitar ataques de tipo MIME sniffing, clickjacking o XSS en integraciones con frontend (CWE-16).

Evidencia:

```
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.14.10
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
# Faltan: X-Content-Type-Options, X-Frame-Options, etc.
```

Recomendacion:

Agregar headers de seguridad en el metodo responder() de app.py:
self.send_header('X-Content-Type-Options', 'nosniff')
self.send_header('X-Frame-Options', 'DENY')

MEDIA **VUL-003 - Exposicion de version del servidor en headers**

Descripcion:

El header 'Server: BaseHTTP/0.6 Python/3.14.10' revela la tecnologia, el framework y la version exacta de Python. Esto facilita el reconocimiento a un atacante que busque vulnerabilidades especificas (CWE-200).

Evidencia:

```
Server: BaseHTTP/0.6 Python/3.14.10
# Presente en TODAS las respuestas HTTP
```

Recomendacion:

Sobrescribir server_version y sys_version en la clase ApiHandler:
server_version = 'API/1.0'
sys_version = ''

BAJA VUL-004 - Ausencia de limitacion de tasa (Rate Limiting)

Descripcion:

La API no implementa ningun mecanismo de rate limiting. Un atacante podria enviar un gran volumen de peticiones para intentar fuerza bruta contra el endpoint /api/privado o causar denegacion de servicio (CWE-770). ThreadingHTTPServer crea un hilo por cada peticion sin limite.

Evidencia:

```
# No se encontro ningun mecanismo de rate limiting
# en app.py ni en api_controller.py
# ThreadingHTTPServer no tiene limite de hilos
```

Recomendacion:

Implementar un rate limiter basado en IP con un diccionario de contadores y timestamps, o usar un reverse proxy (nginx) con limit_req_zone.

BAJA VUL-005 - Sin politica CORS definida

Descripcion:

La API no define headers Access-Control-Allow-Origin. Los navegadores bloquearan peticiones cross-origin por defecto (comportamiento seguro). Sin embargo, la ausencia de una politica CORS explicita puede causar problemas si se integra con un frontend en otro dominio. OPTIONS retorna 405 (CWE-942).

Evidencia:

```
OPTIONS /api/salud -> 405 Method Not Allowed
# No se observa Access-Control-Allow-Origin
```

Recomendacion:

Si se necesita acceso cross-origin, implementar manejo explicito de OPTIONS con headers CORS restrictivos. Si no, documentar como solo same-origin.

INFO VUL-006 - Logging deshabilitado completamente

Descripcion:

El metodo log_message() en app.py esta sobreescrito con 'pass', eliminando todo registro de peticiones. La ausencia total de logs impide detectar ataques, accesos no autorizados o fallos del sistema (CWE-778).

Evidencia:

```
def log_message(self, format, *args):
    pass  # No registrar credenciales
# Archivo: app.py, lineas 30-31
```

Recomendacion:

Implementar logging selectivo que registre metodo, ruta y codigo de respuesta sin incluir headers de autorizacion ni datos sensibles.

INFO VUL-007 - Ausencia de validacion del cuerpo de la peticion

Descripcion:

Aunque la API solo acepta GET, el servidor no limita el tamano del body en metodos POST/PUT. Un atacante podria enviar peticiones con payloads muy grandes que consumirian memoria del servidor antes del rechazo (CWE-400).

Evidencia:

```
POST /api/salud con body de 100,000 bytes -> 405
# El body se lee completo en memoria antes del rechazo
```

Recomendacion:

Implementar un limite de Content-Length en el handler. Rechazar con 413 Payload Too Large si supera 1 KB.

4. ASPECTOS POSITIVOS IDENTIFICADOS

La auditoria tambien identifico buenas practicas de seguridad implementadas:

> Uso de compare_digest() para comparacion de tokens: previene ataques de timing side-channel.
> Validacion correcta del formato Bearer: requiere 2 partes y verifica el prefijo (case-insensitive).
> Binding a 127.0.0.1: la API solo escucha en loopback, no es accesible desde la red externa.
> Whitelist de rutas: solo acepta las 3 rutas definidas; cualquier otra retorna 404.
> Whitelist de metodos HTTP: solo acepta GET; otros metodos retornan 405 con header Allow.
> Header Cache-Control: no-store: previene almacenamiento en cache de respuestas sensibles.
> Content-Type explicito: application/json con charset utf-8 en todas las respuestas.
> Datos ficticios sin base de datos: elimina vectores de SQL injection u ORM injection.
> Serializacion segura con json.dumps(): sin uso de eval(), pickle o deserializacion insegura.
> Separacion clara MVC: la logica de seguridad, datos y presentacion estan bien aisladas.

5. RESULTADOS DE PRUEBAS DINAMICAS

5.1 Pruebas de autenticacion

Prueba	Codigo	Respuesta	Result.
GET /api/privado sin token	401	Token ausente o invalido	PASS
GET /api/privado token correcto	200	auth OK	PASS
GET /api/privado token incorrecto	401	Token ausente o invalido	PASS
GET /api/privado header malformado	401	Token ausente o invalido	PASS

5.2 Pruebas de rutas y metodos

Prueba	Codigo	Respuesta	Result.
GET /api/salud	200	{"status": "OK"}	PASS
GET /api/productos	200	Lista de 3 productos	PASS
GET /api/inexistente	404	Ruta no encontrada	PASS
POST /api/salud	405	Metodo no permitido	PASS
GET /../etc/passwd	404	Ruta no encontrada	PASS

5.3 Pruebas de inyeccion

Prueba	Codigo	Respuesta	Result.
SQL Injection en query string	200	No afecta (sin DB)	PASS
Header injection	401	Token rechazado	PASS
Body POST grande (100KB)	405	Metodo no permitido	NOTA

6. MATRIZ DE RIESGOS

Clasificacion de hallazgos segun probabilidad de explotacion e impacto en el contexto academico.

ID	Vulnerabilidad	Severidad	Probabilidad	Impacto	Riesgo
VUL-001	Token hardcodeado	MEDIA	Alta	Medio	Medio
VUL-002	Sin headers seguridad	MEDIA	Media	Bajo	Bajo
VUL-003	Version expuesta	MEDIA	Alta	Bajo	Bajo
VUL-004	Sin rate limiting	BAJA	Baja	Medio	Bajo
VUL-005	Sin politica CORS	BAJA	Baja	Bajo	Minimo
VUL-006	Logging deshabilitado	INFO	N/A	Bajo	Nota
VUL-007	Sin limite de body	INFO	Baja	Bajo	Nota

7. CONCLUSIONES Y RECOMENDACIONES

La API MVC analizada demuestra un nivel de seguridad adecuado para su proposito academico. Las principales fortalezas incluyen el uso de compare_digest() para prevenir timing attacks, el binding exclusivo a loopback, la whitelist estricta de rutas y metodos, y la ausencia de deserializacion insegura.

Los hallazgos identificados son consistentes con una aplicacion de demostracion y no representan riesgos criticos en el contexto de un laboratorio academico. Se recomienda implementar las siguientes mejoras prioritarias:

> PRIORIDAD 1: Externalizar el token a una variable de entorno (VUL-001)
> PRIORIDAD 2: Agregar headers de seguridad HTTP basicos (VUL-002)
> PRIORIDAD 3: Ocultar la version del servidor (VUL-003)
> PRIORIDAD 4: Implementar logging selectivo sin datos sensibles (VUL-006)
> PRIORIDAD 5: Evaluar rate limiting y CORS segun el caso de uso (VUL-004, VUL-005)

8. ASUNTOS PENDIENTES (Fuera de Alcance)

Los siguientes aspectos no fueron evaluados y requieren analisis adicional:

> Pruebas de carga/estres: evaluar comportamiento bajo alta concurrencia.
> Analisis de dependencias: el entorno virtual (.venv) no fue auditado.
> Configuracion de despliegue: no se evaluo reverse proxy en produccion.
> Pruebas de fuzzing: no se realizaron pruebas de fuzzing automatizado.
> Analisis de API_ACT4: existe otra version de la API que no fue parte de este alcance.

Reporte generado por el Agente de Auditoria de Seguridad
Joaquin - Rakesh Cibersecurity | Tecmilenio
Generado: 2026-11-12 18:35:36